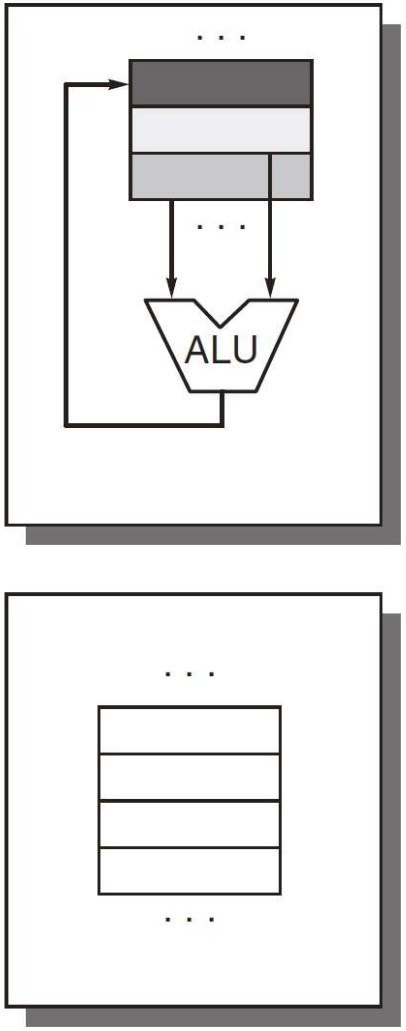
System and Devices (SYS 3)

Lecture 5: Introduction to RISC Pipeline

Dr Pengcheng Liu

Introduction to MIPS

- Microprocessor without Interlocked Pipelined Stages
- 32 registers (32 bit each)
- Uniform length instructions
- RISC-Load store architecture



(d) Register-register/ load-store

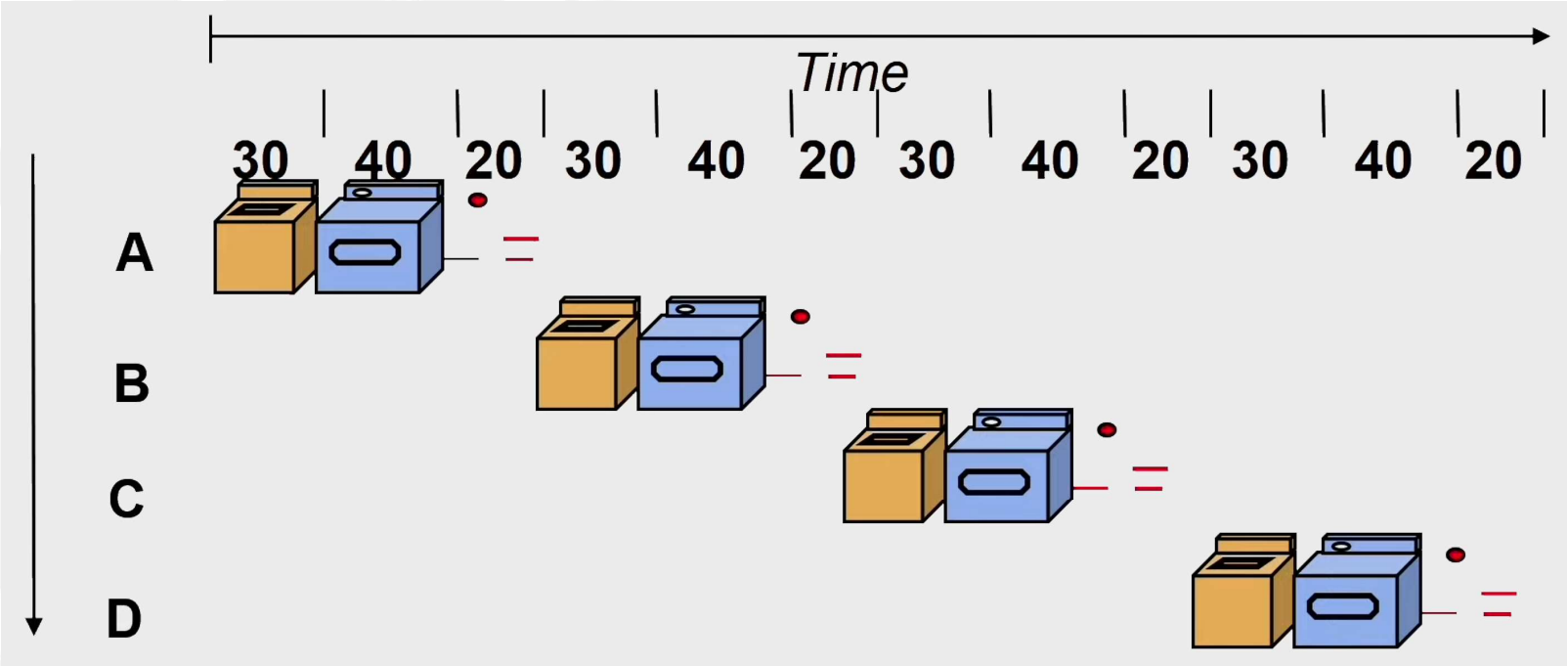
Introduction to MIPS

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R:	op	rs	rt	rd	shamt	funct
I:	op	rs	rt	address / immediate		
J:	op	target address				

op: basic operation of the instruction (opcode)
rs: first source operand register
rt: second source operand register
rd: destination operand register
shamt: shift amount
funct: selects the specific variant of the opcode (function code)
address: offset for load/store instructions ($+/-2^{15}$)
immediate: constants for immediate instructions

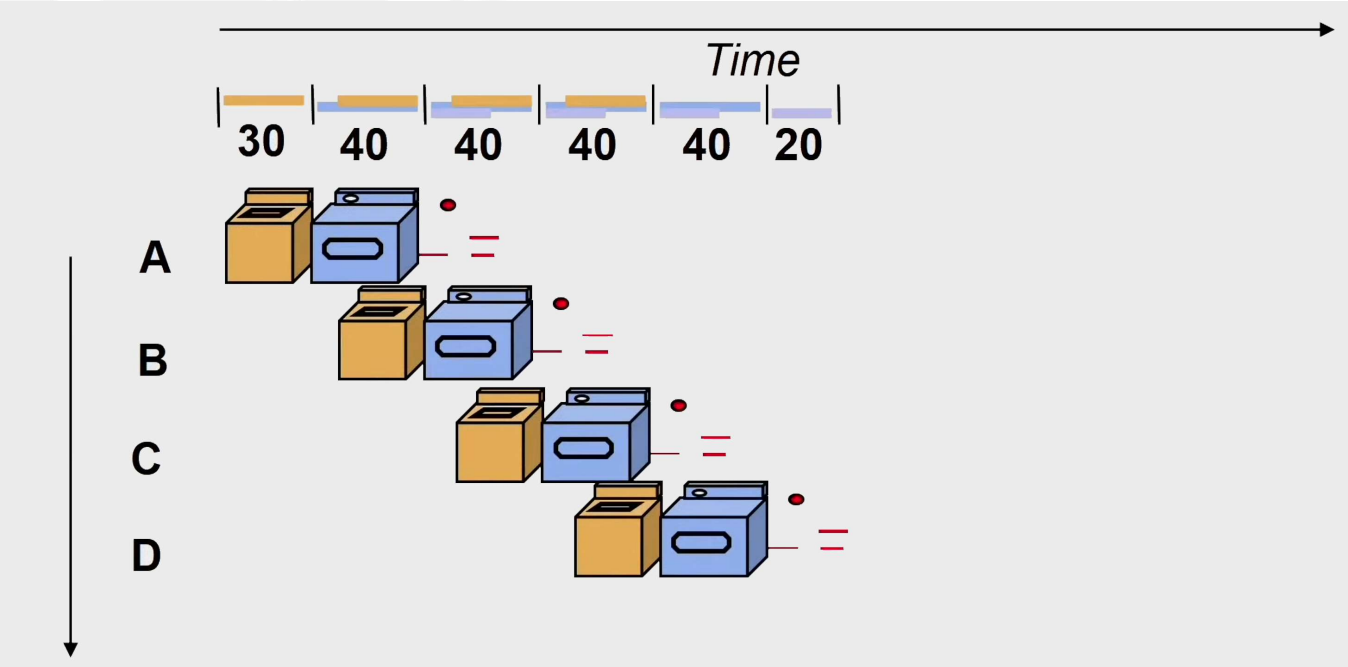
Unpipelined Workflow

- Start work when previous one is fully over
- Sequential laundry takes 6 hours for 4 loads



Pipelined Workflow

- Start work as soon as possible
- Pipelined laundry takes 3.5 hours for 4 loads

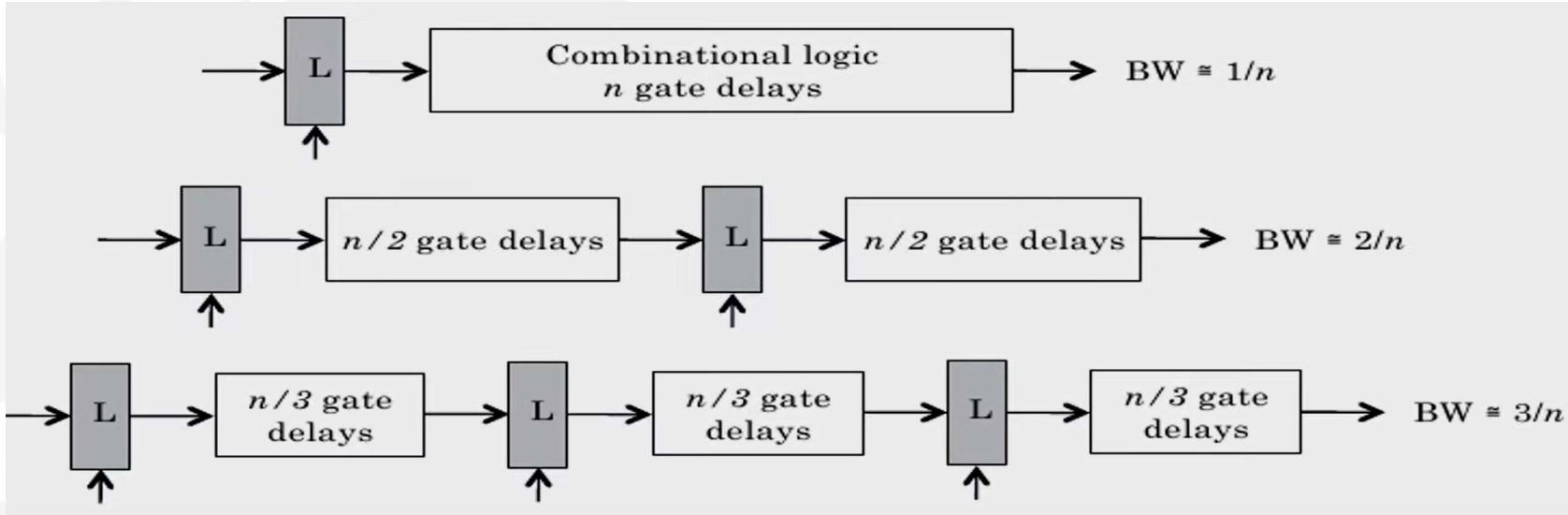


Pipelining Characteristics

- Pipelining doesn't reduce latency of single task, it improves throughput of entire workload
- Pipeline rate limited by slowest pipeline stage
- Potential speedup = Number of pipe stages
- Unbalanced lengths of pipe stages reduces speedup
- Time to fill pipeline and time to drain it reduces speedup

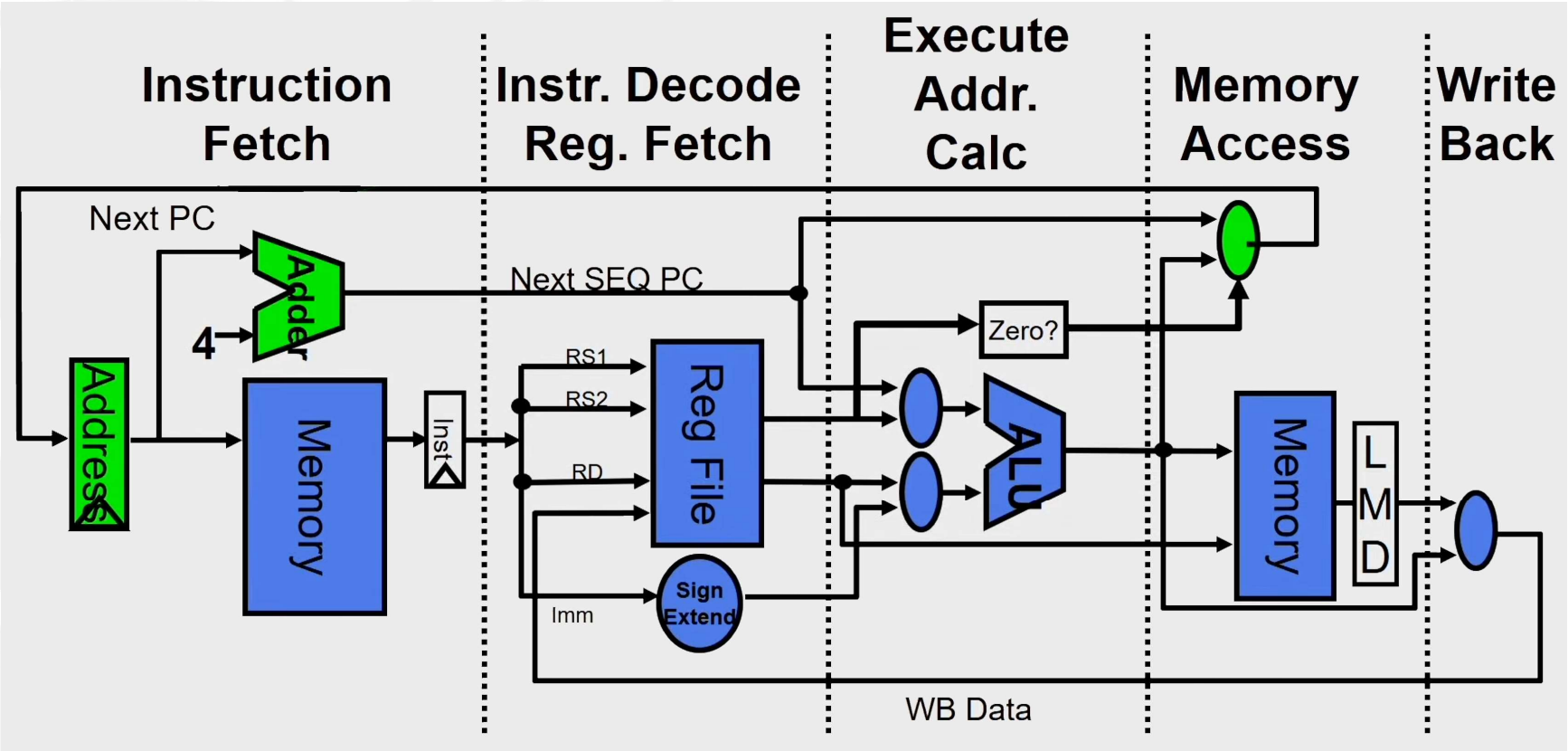
Pipelining in Circuits

- Pipelining partitions the system into multiple independent stages with added buffers between the stages
- Pipelining can increase the throughput of a system

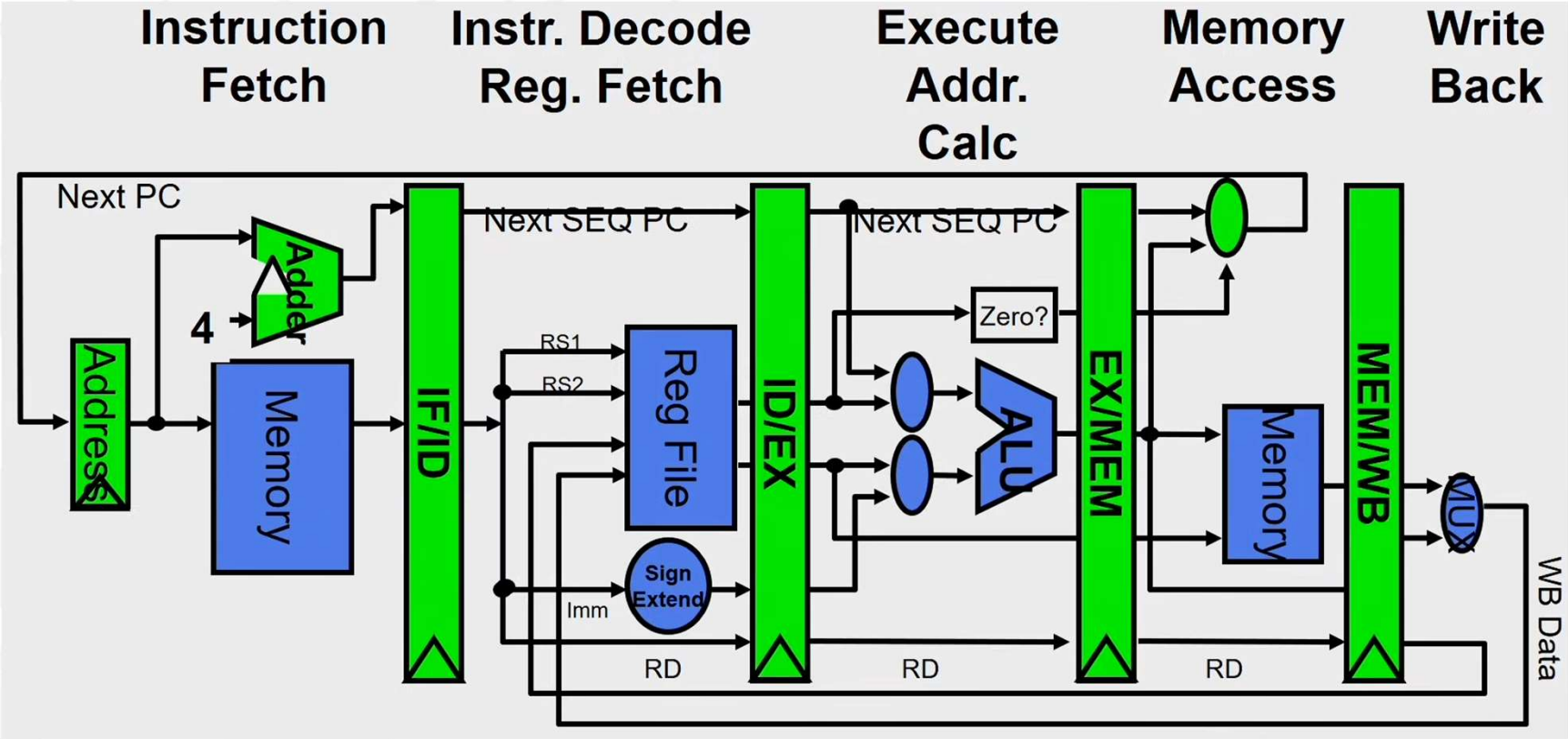


Potential k-fold increase of throughput in a k-stage pipelined system

Unpipelined RISC Data Path



Pipelined RISC Data Path

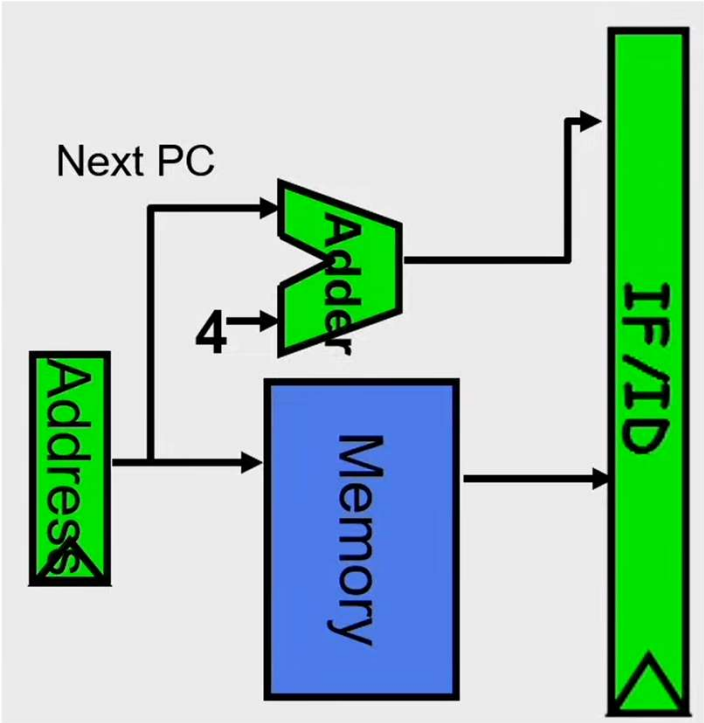


RISC Instruction Pipeline

- Each instruction can take at most 5 clock cycles
- Instruction fetch cycle (IF)

  - Based on PC, fetch the instruction from memory

  - Increment PC



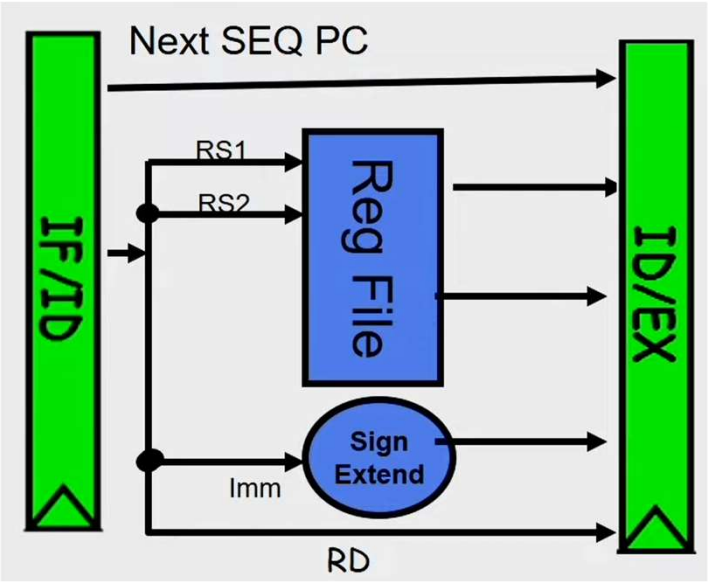
RISC Instruction Pipeline

- Instruction decode/register fetch cycle (ID)
- Decode the instruction + register read operation
- Fixed field decoding

Ex: [ADD R1,R2,R3] : A3.01.02.03

10100011 00000001 00000010 00000011

R1 R2 R3



RISC Instruction Pipeline

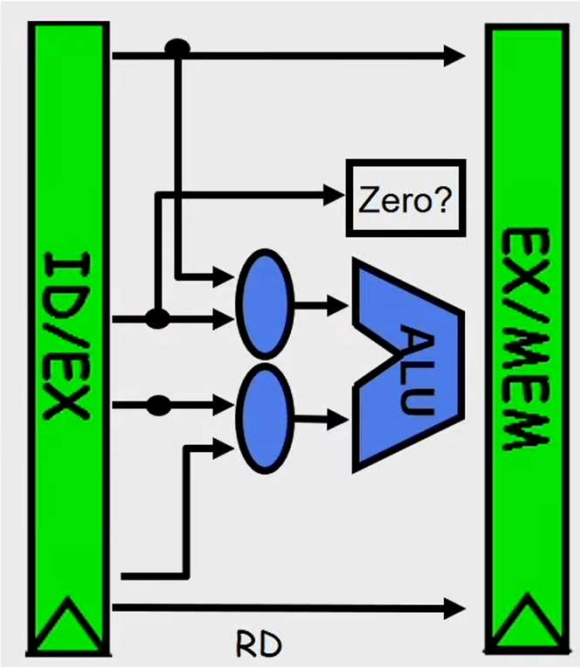
- Execution/Effective address cycle (EX)
- Memory reference: Calculate the effective address

[LW R1,8(R2)] EFF ADDR= [R2] +8

[SW R1,8(R2)]

- Register-register ALU instruction

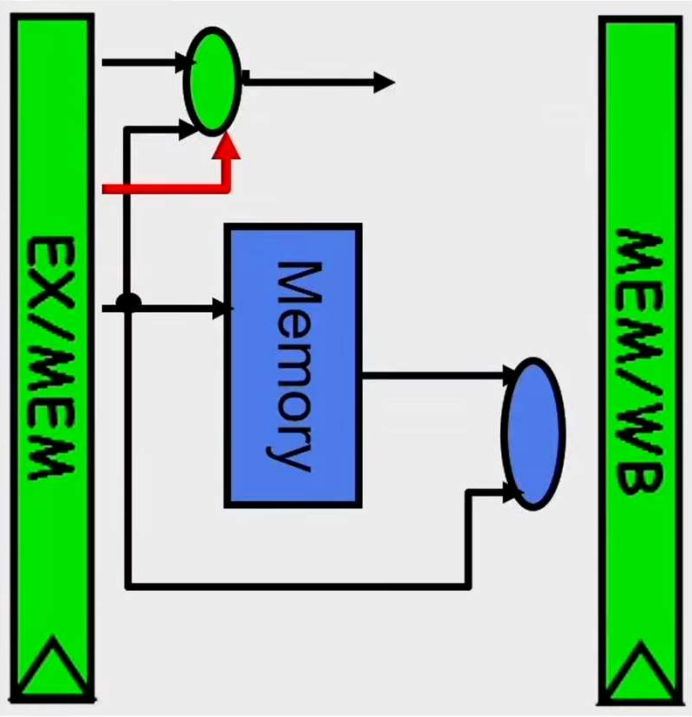
[ADD R1,R2,R2]



RISC Instruction Pipeline

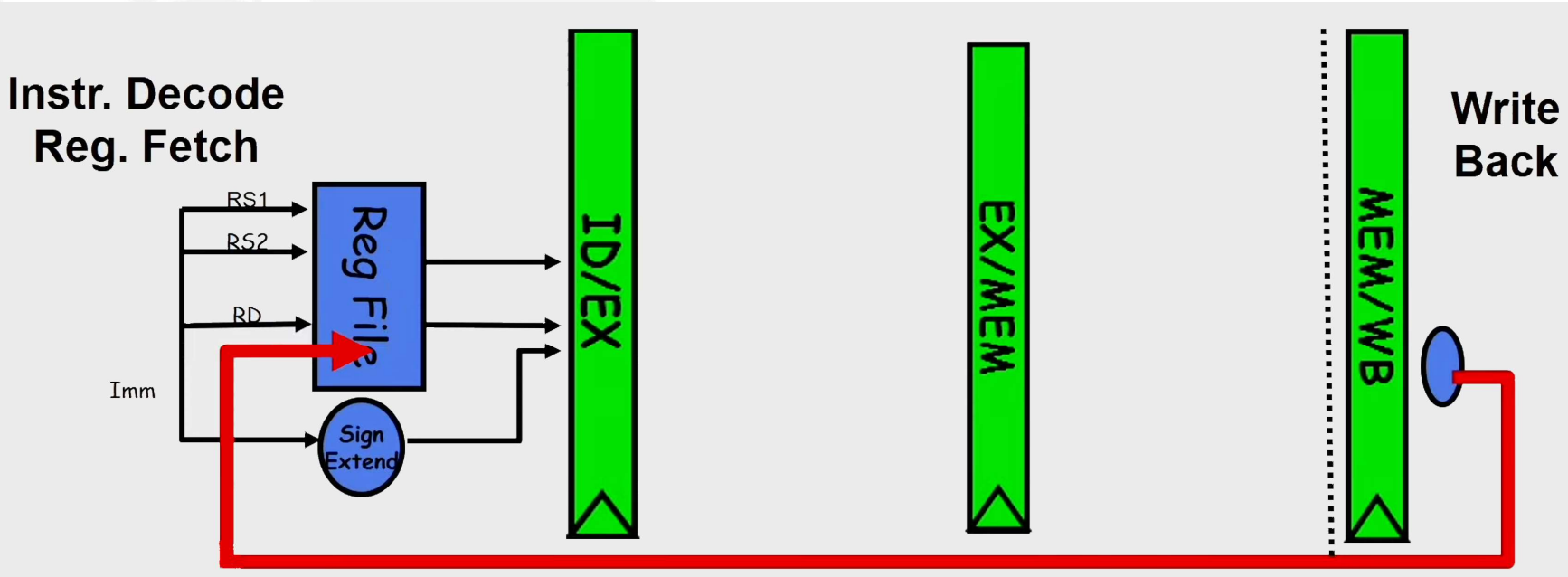
- Memory access cycle (MEM)
- Load from memory and store in register [LW R1,8(R2)]
- Store the date from the register to memory

[SW R3,16(R4)]

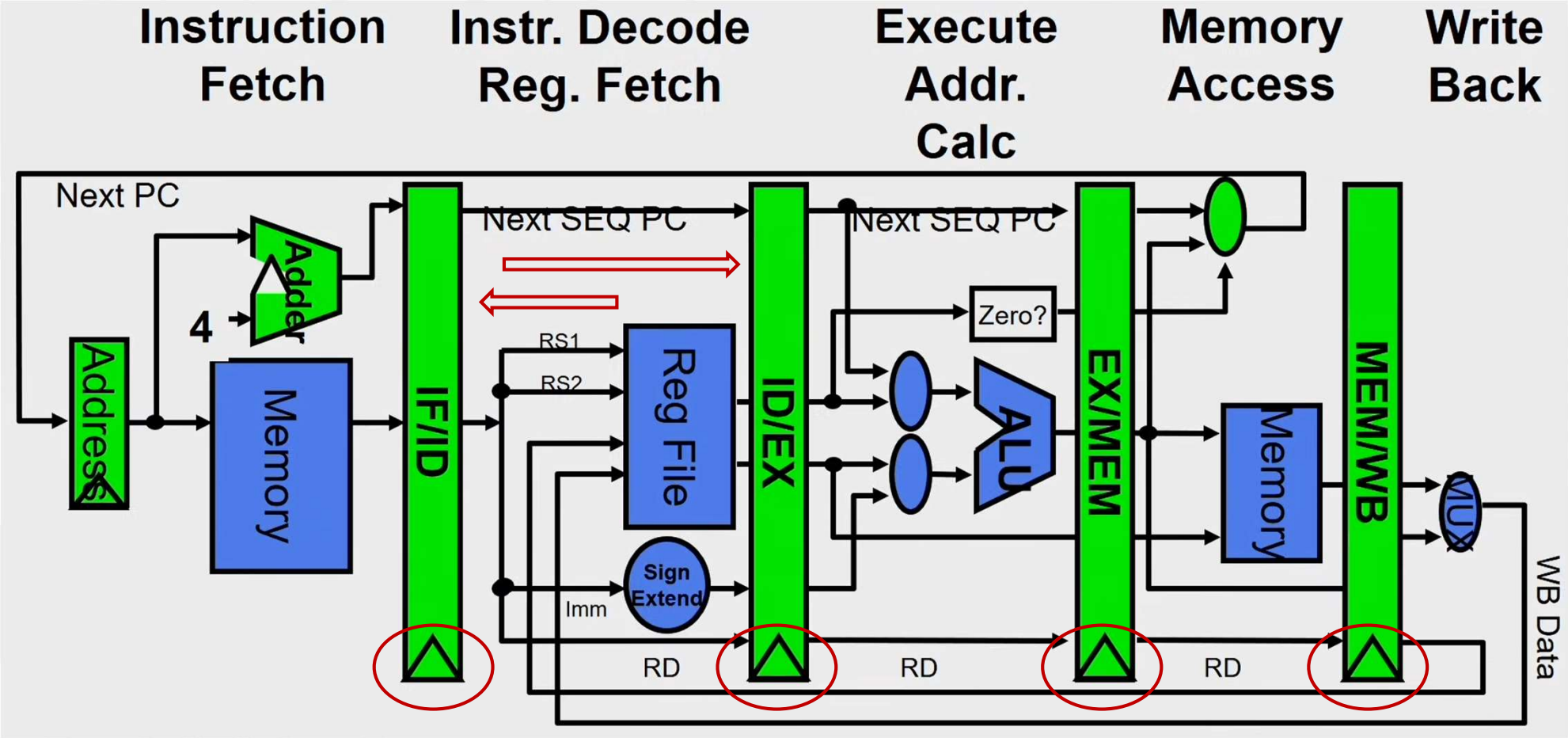


RISC Instruction Pipeline

- Write-back cycle (WB)
- Register-register ALU instruction or load instruction
- Write to register file [LW R1,8(R2)], [ADD R1,R2,R3]



Pipelined RISC Data Path



5 Steps of RISC Data Path

- Each instruction can take at most 5 lock cycles
- Instruction fetch cycle (IF)

  - Based on PC value, fetch the instruction from memory

  - Update PC
- Instruction decode/register fetch cycle (ID)

  - Decode the instruction + register read operation

  - Fixed field decoding

  - Equality check of registers

  - Computation of branch target address if any

5 Steps of RISC Data Path

- Execution/Effective address cycle (EX)

  - Memory reference: Calculate the effective address

  - Register-register ALU instruction

  - Register-immediate ALU instruction

- Memory access cycle (MEM)

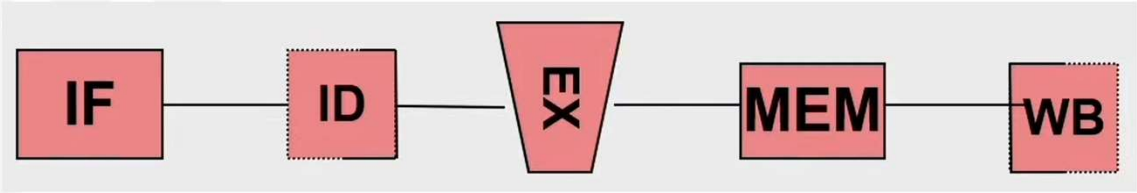
  - Load instruction: Read from memory using effective address

  - Store instruction: Write the data in the register to memory using effective address

5 Steps of RISC Data Path

- Write-back cycle (WB)
  - Register-register ALU instruction or load instruction
  - Write the result into the register file

- Cycles required to implement different instructions
  - Branch instructions: 4 cycles
  - Store instructions: 4 cycles
  - All other instructions: 5 cycles

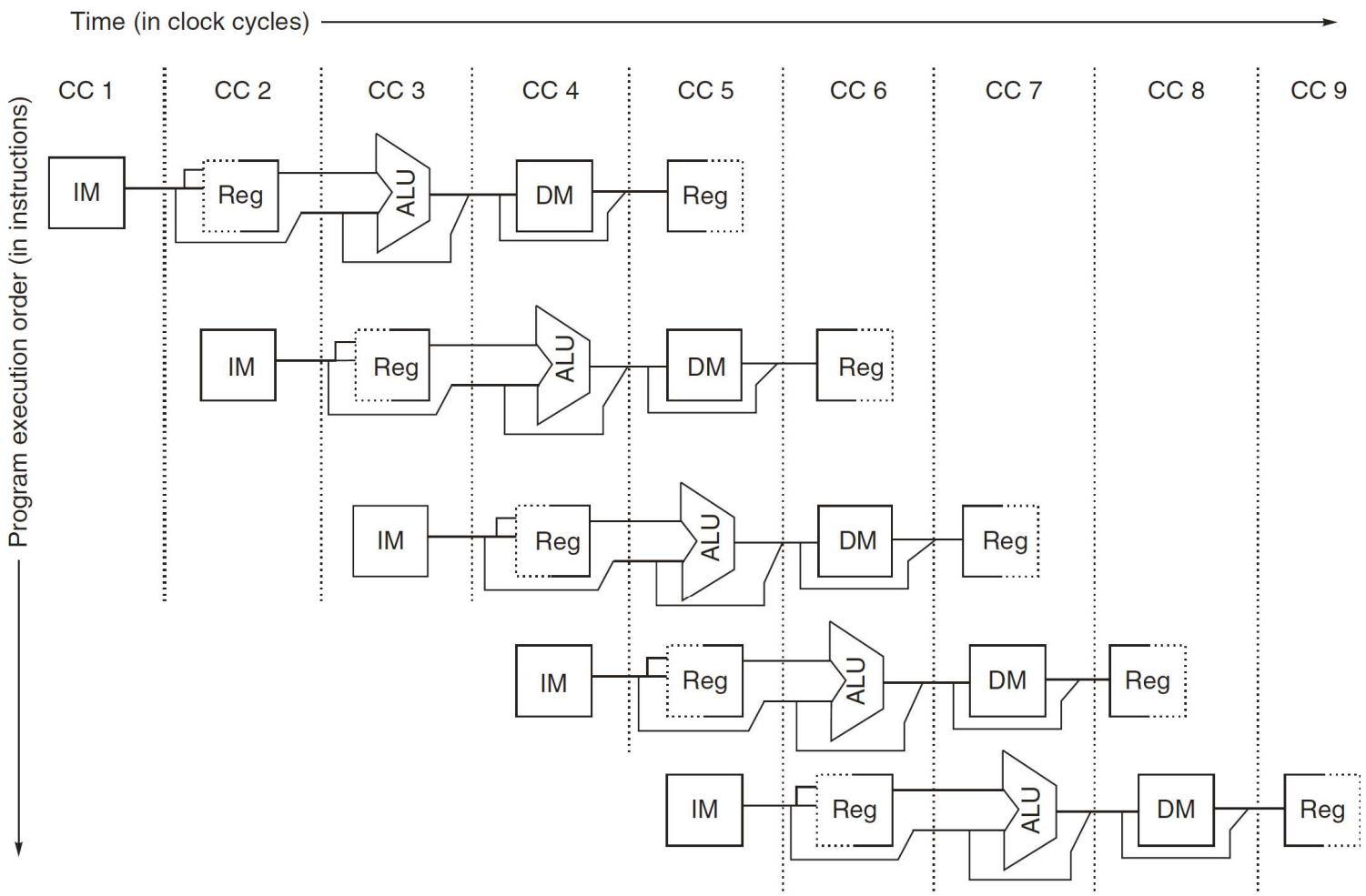


Visualizing Pipelining

	Clock number								
Instruction number	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
Instruction *i*	IF	ID	EX	MEM	WB				
Instruction *i* + 1		IF	ID	EX	MEM	WB			
Instruction *i* + 2			IF	ID	EX	MEM	WB		
Instruction *i* + 3				IF	ID	EX	MEM	WB	
Instruction *i* + 4					IF	ID	EX	MEM	WB

Simple RISC pipeline. On each clock cycle, another instruction is fetched and begins its five-cycle execution. If an instruction is started every clock cycle, the performance will be up to five times that of a processor that is not pipelined. The names for the stages in the pipeline are the same as those used for the cycles in the unpipelined implementation: IF = instruction fetch, ID = instruction decode, EX = execution, MEM = memory access, and WB =write-back.

Visualizing Pipelining



The pipeline can be thought of as a series of data paths shifted in time. This shows the overlap among the parts of the data path, with clock cycle 5 (CC 5) showing the steady-state situation. Because the register file is used as a source in the ID stage and as a destination in the WB stage, it appears twice. We show that it is read in one part of the stage and written in another by using a solid line, on the right or left, respectively, and a dashed line on the other side. The abbreviation IM is used for instruction memory, DM for data memory, and CC for clock cycle.

Pipelining Issues

- **Ideal Case: Uniform sub-computations**

  - The computation to be performed can be evenly partitioned into uniform-latency sub-computations

- **Reality: Internal fragmentation**

  - Not all pipeline stages may have the uniform latencies

- **Impact of ISA**

  - Memory access is a critical sub-computation
  - Memory addressing modes should be minimized
  - Fast cache memories should be employed

Pipelining Issues

- **Ideal Case: Identical computations**

  - The same computation is to be performed repeatedly on a large number of input date sets

- **Reality: External fragmentation**

  - Some pipeline stages may not be used

- **Impact of ISA**

  - Reduce the complexity and diversity of the instruction types

  - RISC architectures use uniform stage simple instructions

Pipelining Issues

- **Ideal Case: Independent computations**

  - All the instructions are mutually independent

- **Reality: Pipeline stalls – cannot proceed**

  - A later computation may require the result of an earlier computation

- **Impact of ISA**

  - Reduce memory addressing modes - dependency detection

  - Use register addressing mode – easy dependencies check

ADD R1,R2,R3 SW R6,16(R1)

Sub R4,R1,R5 LW R8,8(R2)

Summary

Suggested Readings

Computer Architecture: A Quantitative Approach (5th Edition)
by John Hennessy and David A Patterson:

- pp. C-2 – C-11: C.1 Pipelining Introduction